PRD — Petra Paws Pets: Website & Booking System

Project: Petra Paws Pets — Mobile Pet Grooming (Dubai)
Document type: Product Requirements Document (for development)
Stack: Next.js (frontend + API routes) on Vercel
Timeline: 2 weeks (design + development in parallel; backend built alongside design)

---

1. Overview

Petra Paws Pets is a new Dubai-based mobile **dog & cat** grooming business running a single modified grooming van. They have no existing digital presence. We are building a single-page marketing website with an integrated booking system that takes upfront payment, plus an admin panel for the owner to manage the operation themselves.

The booking flow and service model are defined by the provided Figma designs (a 3-step booking modal, desktop + mobile) and are inspired by the Petology mobile-grooming booking flow — but simpler, since Petra Paws is a single van with one groomer (no staff-selection step).

The single most important technical constraint shapes the whole system: **with one van, every appointment is a point on a single daily timeline, and each new booking must leave enough travel time to drive from the previous appointment's location to the new one, and onward to the next.** All booking logic derives from this.

Goals

- A conversion-focused single-page site capturing the brand (luxurious + funky).
- A booking flow where a customer selects a service, location, and an available time slot, then pays upfront to confirm.
- Availability that automatically respects operating hours, days off, travel time between appointments, and owner-defined blackouts.
- A customer-facing way to view booking status / history.
- An admin panel for the owner to manage services & pricing, view & manage appointments, and block off unavailable days/slots.
- Email alerts to the owner on new bookings.

Non-goals

- Multi-van support (single van assumed throughout).
- Products / e-commerce (the SOW notes future "Products" and "Doggy Day Care" sections — out of scope for this build; backend should not preclude adding them later, but nothing is built for them now).
- A full CMS for site content (see §8 — optional, time-permitting only).

---

2. Users & roles

Role	Description	Access
Customer	Pet owner booking a grooming session	Public site, booking flow, own booking lookup
Admin (owner)	Petra Paws operator	Admin panel (auth-protected): services, appointments, blackouts

3. Core booking engine (the heart of the system)

This is the most important and complex part. It must be implemented server-side; the client UI is never the source of truth for availability.

3.1 Operating rules (built-in)

- **Operating hours:** 10:00 to 17:00. The first appointment may start at 10:00; the **last appointment must *end* by 17:00** (i.e. latest start = 17:00 − service duration).
- **Working days:** 6 days/week. **Mondays off.**
- **Service zones:** bookings restricted to defined target areas — Nad Al Sheba, Meydan, and newer Dubai.
- **Pricing:** price is a **matrix of pet type × service × size** (see §3.2a). **5% VAT** is added to the subtotal at the summary step. Pricing is standard across all serviced regions (no per-zone pricing).

3.2 Data model (minimum)

PetType
- Dog, Cat. Selected first in the booking flow.

Service
- `id`, `name`, `description`, `features` (list of strings, e.g. "Bathing", "Blow Dry"), `duration_minutes`, `active` (bool)
- Two services at launch:
- **Basic Grooming** — Bathing, Blow Dry, Nail Clipping, Ear Cleaning, Teeth Brush.
- **Full Grooming** — everything in Basic, plus Full Hair Cut and Scenting & Fluff.
- Editable by admin (name, features, duration, active).

Price *(matrix — see §3.2a)*
- Keyed on `(pet_type, service_id, size)` → `amount`.
- `size` ∈ { Small, Medium, Large } **for dogs only**; `size = null` for cats (cats are flat-priced per service).
- Editable by admin.

Booking
- `id`, `date`, `start_time`, `end_time`, `pet_type`, `service_id`, `size`, `zone_id` (and/or precise customer coordinates/address), `customer_name`, `customer_email`, `customer_phone`, `pet_name`, `pet_breed`, `special_notes`, `subtotal`, `vat`, `total`, `status`, `created_at`, `hold_expires_at`
- `end_time` = `start_time` + service duration.

- `subtotal` = price looked up from the matrix; `vat` = 5% of subtotal; `total` = subtotal + vat.
- `status` ∈ { `pending_payment` , `confirmed` , `cancelled` }.

ServiceZone

- `id` , `name` , `centroid_lat` , `centroid_lng` , `active`
- Used for travel-time/distance calculation and area restriction.

Blackout

- `id` , `date` , `start_time` (nullable), `end_time` (nullable), `reason`
- A full-day blackout (no times) blocks the whole day. A partial blackout blocks a time range. The availability engine treats blackouts exactly like booked time.

3.2a Pricing matrix (verified from Petology live site, 30 Jun 2026)

Price depends on pet type and service — **and on size for dogs only**. Cats are flat-priced per service (no size split). 5% VAT is added on top at the summary step.

Dogs — size × service grid (6 SKUs):

Size	Basic Grooming	Full Grooming
Small	AED 210	AED 249
Medium	AED 275	AED 299
Large	AED 310	AED 349

- Basic = Bathing, Blow Dry, Nail Clipping, Ear Cleaning, Teeth Brush.
- Full = Hair Cut + all Basic features.

Cats — flat per service (NO size):

Service	Price
Cat Basic Grooming	AED 210
Cat Full Grooming	AED 249

> **Important structural note:** the size selector (Small/Medium/Large) applies to **dogs only**. For cats, the booking flow should skip size selection and price flat per service. The data model must allow `size = null` for cats. (Petology handles cat basic-vs-full via an "extras" section — confirm with the client whether Petra Paws mirrors that or simply offers two distinct cat services.)

- **Subtotal** = matrix lookup (dog: type × service × size; cat: type × service).
- **VAT** = 5% of subtotal.
- **Total** = subtotal + VAT, shown on the Review & Confirm step.

> These are Petology's prices, captured for reference. **Confirm Petra Paws' own prices with the client** — they may differ.

3.3 Availability algorithm

When a customer selects **date + service + location**, the server computes valid start times. Availability is **computed dynamically**, never read from a stored fixed grid.

1. **Reject closed days.** If the date is a Monday, or a full-day blackout exists, return no slots.
2. **Build the working window:** 10:00–17:00. Latest valid start = 17:00 − service duration.
3. **Fetch the day's existing `confirmed` and unexpired `pending_payment` bookings, plus blackouts**, sorted by start time. These carve the day into occupied/free segments.
4. **Generate candidate start times** on a fixed **5-minute grid** (e.g. 1:50, 1:55, 2:00, …). This matches the reference platform (Petology), which offers start times at 5-minute intervals. A fine grid lets the engine offer the earliest valid start right after the previous appointment + travel time, rather than wasting van time by rounding up. **Note the distinction:** the 5-minute grid is the *start-time granularity* (what the customer picks from); the *service duration* (~2 hours per the planning note, but stored per-service) is separate and drives the occupied interval + travel-buffer math.
5. **Keep a candidate at time `T` for location `L` only if ALL hold:**
   - **Fits before close:** `T + duration ≤ 17:00`.
   - **No overlap** with any existing booking or blackout interval.
   - **Travel-in satisfied:** `T ≥ previous.end + travelTime(previous.location, L)`. If no previous booking, use `T ≥ 10:00`.
   - **Travel-out satisfied:** `T + duration + travelTime(L, next.location) ≤ next.start`. If no next booking, use `T + duration ≤ 17:00`.
6. **Return surviving candidate times** as the bookable slots.

> A blackout has no location, so for travel purposes treat it as a hard occupied interval that candidates simply cannot overlap; it does not impose a travel buffer of its own.

3.4 Travel-time / distance calculation

- Implement `travelTime(A, B)` as a **single function** so its internals can change without touching the availability logic.
- Use a real distance/drive-time source (e.g. Google Maps Distance Matrix API) keyed on zone centroids or precise coordinates.
- Add a fixed setup buffer (e.g. +10 min for parking/setup) on top of raw drive time.
- Cache results between known zone pairs to limit API calls (zone pairs are few and stable).

3.5 Area restriction (CONFIRMED: dropdown)

Area restriction is done via an **area dropdown** — the customer selects their area/neighbourhood from a list of allowed zones (Nad Al Sheba, Meydan, newer Dubai). Areas outside the served list are not selectable / are blocked with a clear message. (Geofencing is not used.)

3.6 Booking write & payment (race-condition safety)

Upfront payment is required to secure a booking. Sequence:

1. Customer selects a valid slot → server **re-validates that exact slot against current data** (never trust the client's slot list) → creates a `Booking` with `status = pending_payment` and a short hold TTL (e.g. `hold_expires_at = now + 10 min`).
2. The `pending_payment` hold blocks that slot (and its travel implications) for other customers.
3. Customer pays via the payment gateway:
   - **Success** → `status = confirmed`, trigger admin email alert.
   - **Failure / timeout** → hold expires, slot is released automatically.
4. Expired `pending_payment` holds must be ignored/cleaned (cron or lazy check at read time).

Critical: because each booking affects its neighbours' travel feasibility, always re-check the three constraints (§3.3 step 5) against *currently held/confirmed* bookings at the moment of hold creation.

3.7 Payment gateway (CONFIRMED: Ziina)

- Upfront payment required to confirm (per SOW).
- **Payment gateway: Ziina** (ziina.com) — UAE provider, licensed by the Central Bank of the UAE. Integrate Ziina's payment-gateway API for the upfront online payment on the Review & Confirm step. Developer docs: https://docs.ziina.com/getting-started.
- Supports card / Apple Pay / Google Pay online checkout.
- Transaction fees are the client's cost (per SOW exclusions).

---

4. Public website (single page)

Per the SOW, a single-page site with these sections:

- **Home / hero:** catchy visual banner, clear intro to the mobile grooming concept, immediate booking CTA. Vibe: luxurious + funky (reference points: premium feel of Pride in Grooming + fun energy of Shampooch).
- **Services & pricing:** clear breakdown of **cat and dog** grooming options (Basic Grooming, Full Grooming) with their feature lists and prices (dogs priced by size; cats flat per service). Includes a **Dog / Cat toggle** ("Tailored Attention for Every Fur and Foot" section that switches between dog and cat services). Driven by the admin-editable Service records and pricing matrix (§3.2a).
- **Booking:** entry point to the booking flow (§3, §5).
- **Mobile-first:** the booking process must be seamless on mobile.

A "Browse Products" CTA may be included as a placeholder/hidden element for the future products phase, but no product functionality is built now.

---

5. Customer booking flow (UX)

A 3-step modal, opened from the main CTA. Matches the provided Figma (desktop + mobile). Step indicator (1-2-3) at the top.

Step 1 — Choose Your Service
- **Pet type** toggle: Dog / Cat.
- **Service** selection: Basic Grooming or Full Grooming (each shows its feature list).
- **Pet size:** Small / Medium / Large — **shown for dogs only**. For cats, skip the size selector (cats are flat-priced per service).
- Price shown updates from the matrix per the selected combination.
- Continue.

Step 2 — Tell Us About Your Pet / Pet & Appointment Details
- Pet name, pet breed, owner name, phone number.
- **Area / neighbourhood** (dropdown or address, per §3.5 decision). If outside serviced zones → blocked with a clear message.
- **Preferred date** → **preferred time** (available times computed via §3.3).
- Special notes (optional).
- Continue. (Back to Step 1 available.)

Step 3 — Review & Confirm
- Booking summary: service, pet type + size, pet name, owner, phone, location, date, time.
- **Subtotal**, **VAT (5%)**, **Total Amount**.
- **Make Payment** → upfront payment → booking confirmed.
- (Back to Step 2 available.)

On success: confirmation (booking reference, details) + confirmation email; owner receives a new-booking alert.

> Note: the Figma summary shows trust microcopy ("Replies in ~5 min", "We come to you", "Free rescheduling"). The rescheduling/cancellation policy is still pending (§10) — payment is upfront, so this affects refunds.

---

6. Customer booking status / history

Customers need to see the status/history of their bookings. **Recommended approach: email + booking-reference lookup with a magic link — no passwords, no account creation.** This is the right fit for a single-van grooming business: low friction for customers (most book once or occasionally), nothing for the owner to manage, and no password security surface to maintain.

How it works:

- On confirmation, the customer gets a **unique booking reference** and a **confirmation email containing a secure magic link** to a status page for that booking.
- The status page shows booking details and current status (`confirmed` / `cancelled` , upcoming vs. past).
- A **lookup page** (reference code + email) lets a returning customer retrieve a booking if they don't have the email handy. The magic link in the email is the primary path; the code+email lookup is the fallback.
- If multiple bookings exist for an email, the status page shows a simple **history list** of that email's bookings.

What we can also support (optional, low effort, worth offering the client):

- **WhatsApp confirmation** in addition to email — the Granola note and Figma both reference WhatsApp ("Booking was easy on WhatsApp"), and it's the dominant channel in the UAE. A confirmation/status link sent via WhatsApp would likely get better engagement than email alone. Flag as a nice-to-have.
- **Self-service cancellation/reschedule from the status page** — only if the cancellation/rescheduling policy is confirmed (currently pending, see §10). The page is the natural home for it once policy is decided.

> Deliberately **not** building full authenticated customer accounts — more than this business needs, and adds a password/security surface for little benefit. The magic-link + reference-code model satisfies the requirement cleanly.

---

7. Admin panel

Auth-protected area for the owner. This roughly doubles the build surface beyond the booking engine — see §9 scope note.

7.1 Authentication

- Single admin login (email + password, or email magic link). Minimal — one operator.

7.2 Services & pricing management *(requested item 1)*

- Create / edit / deactivate services (name, description, feature list, duration, active status).
- Edit the **pricing matrix** — a grid of prices keyed on pet type × service × size (Dog/Cat × Basic/Full × S/M/L). This is a grid editor, not a single price field.
- (VAT rate of 5% can be a config value; confirm whether the client needs it editable.)
- Changes reflect immediately on the public Services & Pricing section and in the booking flow.

7.3 Appointment management *(requested item 2)*

- List view of all appointments (upcoming and past), filterable by date/status.
- View full booking detail (customer, pet, service, location, payment status).
- Cancel an appointment (frees the slot).
- Daily/agenda view showing the day's route order is desirable (helps the owner see the timeline the engine built).

7.4 Blackout / no-service management *(requested item 3)*

- Add a **full no-service day** (e.g. sick day, van maintenance, personal).
- Add a **partial block** (specific time range on a date) when the owner can't operate part of a day.
- Remove blackouts.
- The availability engine treats blackouts identically to booked time (§3.3).

7.5 Email alerts

- On every new **confirmed** booking, send an email alert to the owner with booking details.
- (Optional, time-permitting) email the owner on customer cancellations.

---

8. Content management *(requested item 4 — OPTIONAL)*

Owner-editable site copy/images (hero text, about, imagery) is **out of scope unless time permits**. A CMS is the most time-expensive and least essential piece for a single-page site — site copy can be developer-set for launch. Explicitly deferred so it does not eat the core timeline.

---

9. Scope & priority

To set timeline expectations clearly. The admin panel (§7) is a meaningful build on top of the core engine; the priority order below should drive the 2-week sprint.

Must-have (core deliverable)

1. Single-page website (Home, Services & Pricing, Booking) — mobile-first.
2. Core booking engine: availability with operating rules + travel-time buffers + zone restriction.
3. Upfront payment + booking confirmation (with race-condition-safe holds).
4. Customer booking status / history lookup.
5. Admin: appointment management (§7.3).
6. Admin: services & pricing management (§7.2).
7. Admin: blackout / no-service management (§7.4).
8. Email alerts to owner on new bookings.

Optional (only if time permits)

1. Content management (§8).
2. Cancellation emails; admin route/agenda day view.

---

10. Decisions

Confirmed

1. **Stack:** Next.js + Vercel.
2. **Area restriction:** area dropdown (not geofencing) — §3.5.
3. **Payment gateway:** Ziina (ziina.com) — §3.7.
4. **Customer lookup:** email + booking-reference with magic link (no accounts) — §6.
5. **Slot granularity:** 5-minute start-time grid, matching the reference platform; service duration stored per service (~2 hours) — §3.3.

6. **Pricing:** taken from the reference booking platform (Petology) — dogs priced by size, cats flat per service — §3.2a.

Pending (do not block build; owner confirming)

1. **Cancellation policy:** can customers cancel themselves, or admin-only? Affects refunds (payment is upfront) and the status page (§6). Build the status page so cancellation can be enabled once decided.
2. **Rescheduling:** whether self-service rescheduling is in scope (Figma shows "Free rescheduling"). Affects booking flow + status page + admin.
3. **Pricing confirmation:** prices in §3.2a are the reference platform's; confirm Petra Paws' own final numbers before launch.
4. **VAT:** confirm whether the 5% rate needs to be admin-editable or can be a fixed config value.

---

11. Notes & constraints

- **Single van** throughout — no van-assignment logic; the day is one sequential timeline.
- **Server is source of truth** for availability — always re-validate before creating a hold.
- `travelTime()` **is the one swappable dependency** — isolate it.
- Plugin subscriptions, payment gateway transaction fees, hosting, and domain are the **client's** costs (per SOW).
- Design: max 2 iterations (per internal planning).